HDOC Day-52

****

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named previous_greater() using its address. Use pointers to pass the array between the calling function and previous_greater(). The function previous_greater() must:

- Find the previous greater element for each element in arr[].
- Search only the elements to the left of the current element and select the nearest element that is greater than the current element.
- Display -1 if no greater element exists to the left.
- Display the results in the order of their appearance in the array, separated by commas.
- Use a nested-loop approach to solve the problem.

1. Problem Understanding

The program reads the size and elements of a variable-length integer array. It passes the array to previous_greater() using its address. For every element, the function searches only to its left and selects the nearest element that is greater. If no such element exists, it displays -1. The results are displayed in array order, separated by commas.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Expected Output
1	4	4 5 2 25	-1, -1, 5, -1
2	4	13 7 6 12	-1, 13, 7, 13
3	4	1 3 2 4	-1, -1, 3, -1
4	5	1 2 3 4 5	-1, -1, -1, -1, -1
5	5	5 4 3 2 1	-1, 5, 4, 3, 2

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length integer array arr[n].
4. Read n array elements.
5. Pass the array address and size to previous_greater().
6. Display the previous greater element for each array element.
7. Stop.

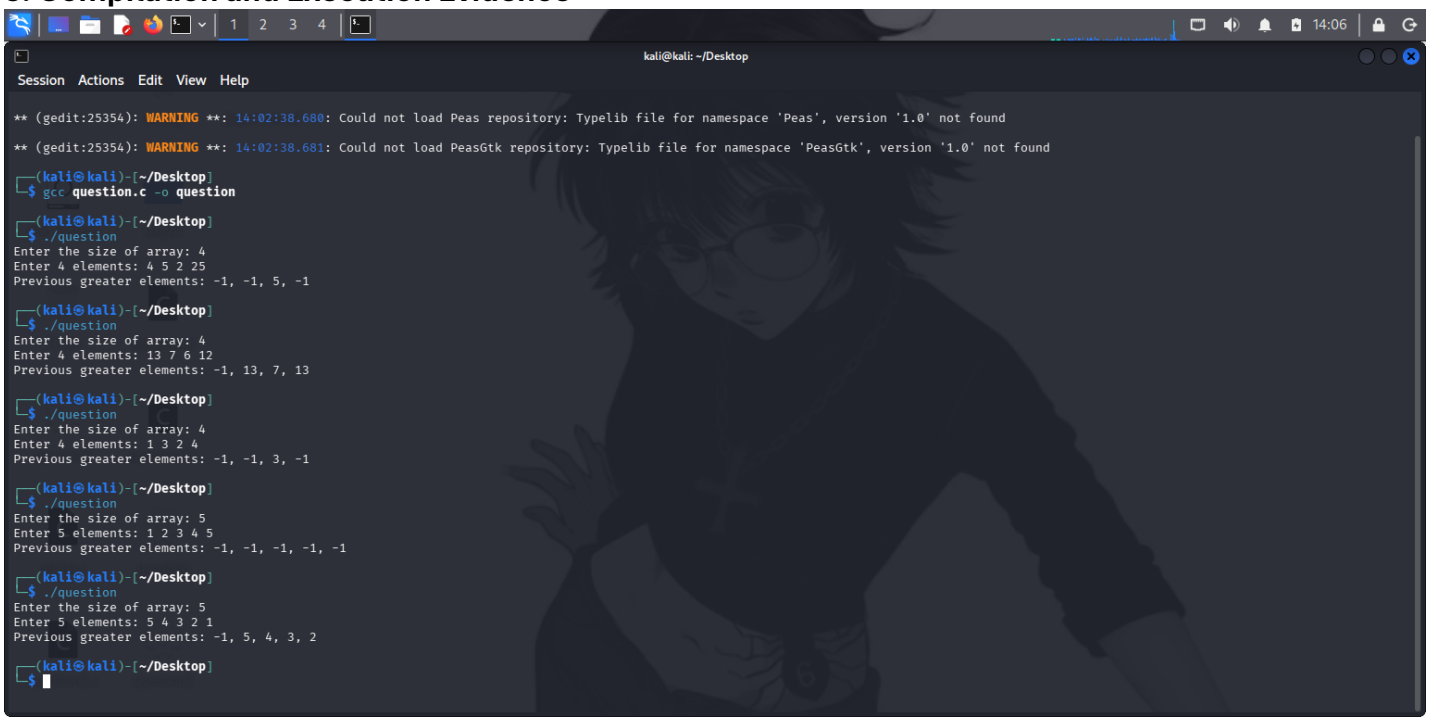
Function: previous_greater()

1. Traverse each element from left to right.
2. Set previous greater as -1 for the current element.
3. Using a nested loop, search from the immediately previous element towards the left.
4. If a greater element is found, store it and stop the inner loop.
5. If none is found, keep -1.

4. C Program

```c
#include <stdio.h>
void previous_greater(int *arr, int n)
{
   int i, j, previous;
   for(i = 0; i < n; i++)
   {
      previous = -1;
      for(j = i - 1; j >= 0; j--)
      {
         if(*(arr + j) > *(arr + i))
         {
            previous = *(arr + j);
            break;
         }
      }
      printf("%d", previous);
      if(i < n - 1)
         printf(", ");
   }
   printf("\n");
}
int main()
{
   int n, i;
   printf("Enter the size of array: ");
   scanf("%d", &n);
   int arr[n];
   printf("Enter %d elements: ", n);
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);
   printf("Previous greater elements: ");
   previous_greater(arr, n);
   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	Actual Result	Status
1	4	4 5 2 25	-1, -1, 5, -1	Pass
2	4	13 7 6 12	-1, 13, 7, 13	Pass
3	4	1 3 2 4	-1, -1, 3, -1	Pass
4	5	1 2 3 4 5	-1, -1, -1, -1, -1	Pass
5	5	5 4 3 2 1	-1, 5, 4, 3, 2	Pass